



Guía de Evaluación: Semana 3 - DevOps, Testing Multinivel y Ecosistema (Mid Level)

Taller: Contenerización, CI/CD y Calidad Continua **Tema:** "En mi máquina funciona ya no es una excusa válida."

1. Objetivo de la Evaluación

Validar la adopción de un **Mindset de QA y DevOps** por parte del estudiante, integrando la inmutabilidad de la infraestructura con una estrategia de pruebas madura. Se evaluará su capacidad para orquestar a la IA en la generación de contenedores seguros y pipelines automatizados, asegurando que apliquen correctamente los **7 Principios del Testing**, diferencien los niveles de pruebas (Componente e Integración) y apliquen técnicas de Caja Blanca y Caja Negra en un flujo automatizado y versionado.

2. El Reto: "El Pipeline Inquebrantable y Multinivel"

Los equipos deben sacar su aplicación de sus entornos locales y empaquetarla en Docker. Su reto principal es construir un pipeline en GitHub Actions que diferencie visualmente la ejecución de pruebas de **Componente** y de **Integración**. Además, deben implementar al menos una prueba de **Caja Negra** (ej. simulando la creación de un usuario desde la API) ejecutándose *dentro* del contenedor orquestado. Todo el proceso debe culminar en un release ordenado usando GitFlow.

3. Conceptos Clave a Evaluar (El "Human Check" DevOps)

Para aprobar, el equipo debe evidenciar dominio sobre estos conceptos durante su defensa y en el código:

- **Estrategia Multinivel, Tipos de Pruebas y Documentación (MD):** Creación de un archivo Markdown (ej. `TEST_PLAN.md`) estructurado como un **informe técnico formal** para mostrar los *Test Plan* y *Test Cases*. En este documento deben debatir y justificar teóricamente cómo aplican **Los 7 Principios de las Pruebas**, su **Estrategia Multinivel** y los **Tipos de Pruebas**. A nivel práctico, esto se complementa con la distinción en el

pipeline entre pruebas de Componente (aisladas) y de Integración (comunicación de puertos), e implementación funcional de Caja Blanca y Caja Negra.

- **Los 7 Principios de las Pruebas:** Capacidad para argumentar sus decisiones de testing basándose en principios como la "Paradoja del Pesticida" y "Las pruebas dependen del contexto" (Principio 6).
- **Inmutabilidad y Seguridad (Docker):** Imágenes base ligeras, ejecución sin privilegios de root, y uso de herramientas no funcionales (ej. Docker scan) para detectar vulnerabilidades en la imagen.
- **Shift-Left Quality (Pipeline CI/CD):** Bloqueo de integraciones defectuosas *antes* de llegar a la rama principal mediante reportes automatizados.

4. Entregables Obligatorios

1. **Infraestructura como Código:** Archivo **Dockerfile** optimizado y seguro en la raíz del repositorio.
2. **Definición del Pipeline:** Archivo YAML en **.github/workflows/** que diferencie visualmente (ej. *jobs* distintos) las pruebas de componente de las de integración.
3. **Plan de Pruebas como Informe (Archivo MD):** Archivo Markdown (ej. **TEST_PLAN.md**) redactado y estructurado como un informe profesional. Debe contener el diseño claro de los Test Plan y Test Cases, y la argumentación teórica solicitada sobre los 7 Principios y la Estrategia Multinivel.
4. **Evidencia de Ejecución Multinivel:** Enlace directo o captura del pipeline en verde, demostrando la ejecución exitosa de pruebas (Caja Negra y Caja Blanca), linter, análisis de vulnerabilidades de imagen y build.
5. **Flujo GitFlow y Release:** Evidencia en el repositorio del uso estricto de GitFlow, incluyendo Pull Requests documentados para el paso de la rama **develop** a **main** (Release).

5. Rúbrica de Evaluación

Criterio	Nivel Experto (5.0) - Senior Track	Nivel Competente (3.5) - Mid Track	Nivel Insuficiente (2.0) - Junior Track
----------	------------------------------------	------------------------------------	---

Infraestructura Inmutable y CI/CD	Dockerfile impecable (<i>Multi-stage</i> , ligero, no-root, escaneo de vulnerabilidades efectivo). Pipeline modular que separa <i>Jobs</i> de Componente e Integración, muestra reportes detallados y bloquea merges (Branch Protection) si hay fallos.	Dockerfile funcional con imagen estándar. El pipeline se ejecuta exitosamente en cada PR, aunque corre todas las pruebas en un solo paso y no cuenta con escaneo avanzado de vulnerabilidades.	Imagen muy pesada (ej. copia <code>node_modules</code> local) o corre como root. Pipeline inexistente, comentado o de activación exclusivamente manual. Sin reportes en GitHub Actions.
Testing Multinivel y Técnicas	Evidencia clara de pruebas de Caja Blanca y al menos una de Caja Negra (ej. flujo API real). Diferencia técnica y física entre Unit/Component e Integration.	Tiene pruebas que pasan en el CI, pero están mezcladas. La prueba de "Caja Negra" conoce detalles internos de implementación.	Carece de pruebas funcionales (Caja Negra). Solo hay pruebas unitarias triviales o saltadas en el pipeline.
Plan de Pruebas como Informe (TEST_PLAN.md)	Presenta un informe técnico impecable. Detalla exhaustivamente los <i>Test Suites</i> , <i>Test Plan</i> y <i>Test Cases</i> . Justifica con maestría los 7 Principios, la Estrategia Multinivel y Tipos de Pruebas aplicados en el proyecto.	El documento existe con formato de informe e incluye componentes básicos (Suites/Cases). Menciona la teoría (Principios y Estrategia) pero la argumentación carece de profundidad y detalle aplicado al proyecto.	No existe el archivo, solo tiene notas desordenadas sin formato de informe. Faltan los <i>Test Cases</i> claros o no hay justificación teórica de los 7 principios de las pruebas.
GitFlow y Estrategia de Release	Uso immaculado de GitFlow. Existe un PR formal de <code>develop</code> a <code>main</code> (Release) que agrupa las features, dispara el pipeline y	Se observan ramas de feature y un merge hacia <code>main</code> , pero carece de formalidad de Release o	Commits directos a <code>main</code> o <code>develop</code> . No hay estrategia de ramas, Pull Requests ni

	versiona (tag) el código y artefacto.	versionado semántico.	versionado.
Human Check y Validación HITL	Argumentación técnica profunda oral. Identifican alucinaciones de la IA al generar pruebas/Dockerfile. Explican claramente qué delegaron a la IA y cómo auditaron sus respuestas.	Entienden los principios básicos de Docker y CI/CD, pero dudan al diferenciar pruebas de integración verdaderas vs. dependencias mockeadas en su propio código.	Copian/pegan el YAML y Dockerfile de la IA. No saben responder principios de pruebas ni explican el código entregado.

6. Formato de Presentación ("Auditoría del Ecosistema")

Los equipos se someterán a una auditoría técnica en vivo (10 minutos por equipo):

1. **Defensa Teórica y Presentación del Informe:** * El equipo debe abrir y proyectar su archivo `TEST_PLAN.md`.
 - Deberán mostrar cómo estructuraron su reporte .
 - El instructor preguntará basándose en el documento: *"Expliquen cómo es el diseño actual de su pipeline y su plan de pruebas, y cómo justifican sus decisiones bajo el Principio 6 (Contexto)."*
2. **La Prueba del Pipeline y Niveles:** * Se mostrará el YAML del pipeline y su última ejecución exitosa.
 - El equipo deberá señalar dónde se ejecuta la prueba de **Caja Negra** y explicar por qué la consideran así.
3. **El "Human Check" de Infraestructura y Pruebas (4 min):**
 - *Comprensión:* El instructor tomará una prueba de integración generada por el equipo/IA y preguntará: *"¿Es esto realmente una prueba de integración, o es una prueba unitaria?"*
 - *Calidad:* Validar si incluyeron pruebas no funcionales, preguntando: *"¿Cómo detectan vulnerabilidades en la imagen Docker generada?"*
 - **Validación HITL:** Interrogatorio crítico. El estudiante debe explicar línea a línea las instrucciones que la IA generó para asegurar que existió un criterio humano y técnico superior al momento de aceptar dicho código.